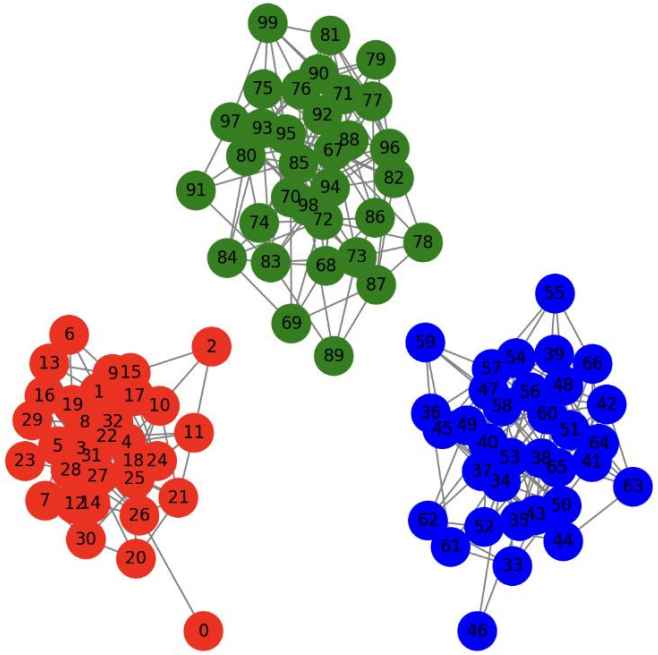
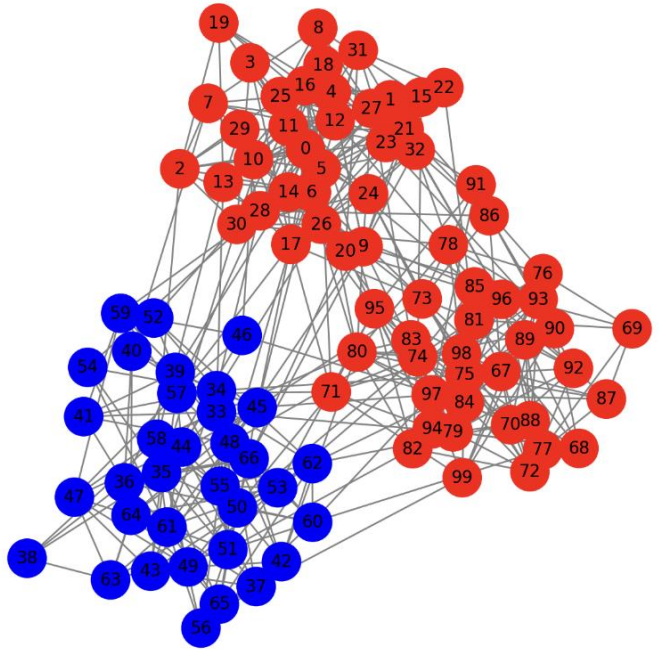
S13: Community Detection

Communities

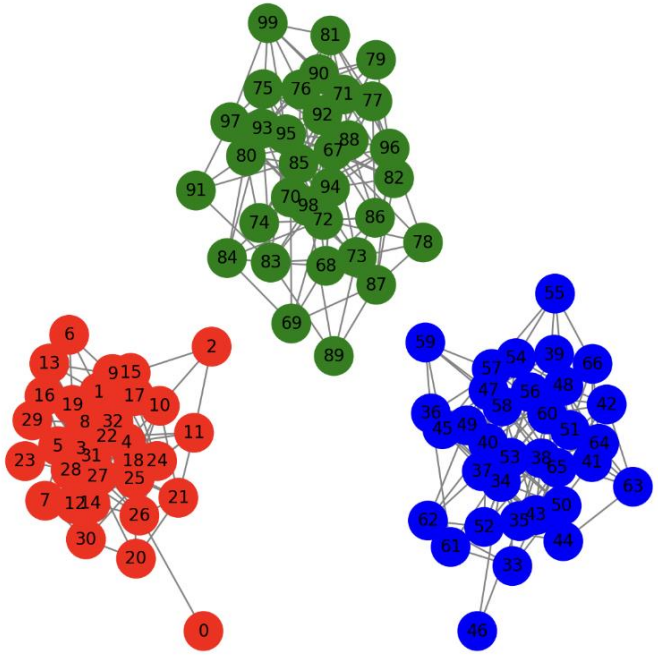
- Clusters of nodes that are more densely connected internally than with the rest of the network
- Disconnected communities: components
- Clustering: the sub-network contains such a community



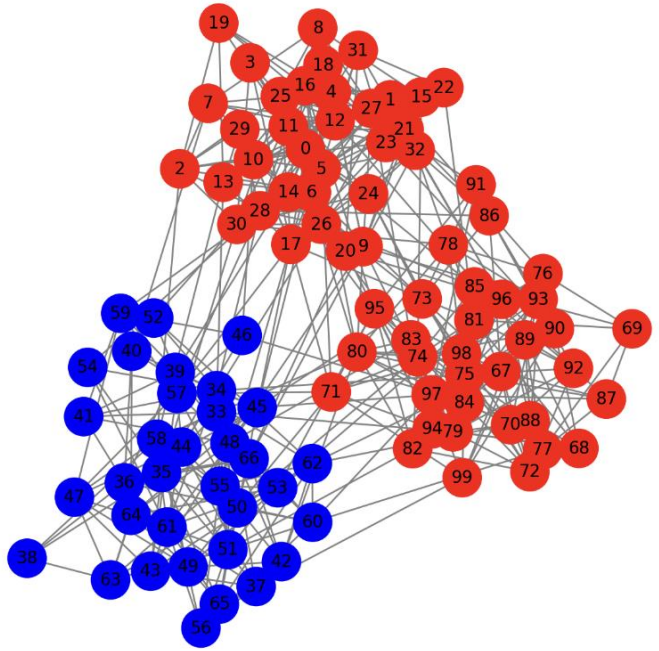


Detect communities

- Isolate/identify each community



- Helps in understanding network organization
  - Identifies meaningful groupings that may not be obvious
  - Analyze how information, behaviors, or diseases propagate
  - Allows optimizing infrastructure, traffic flow, or communication systems



Real-world applications

Improving recommendation systems to suggest relevant content

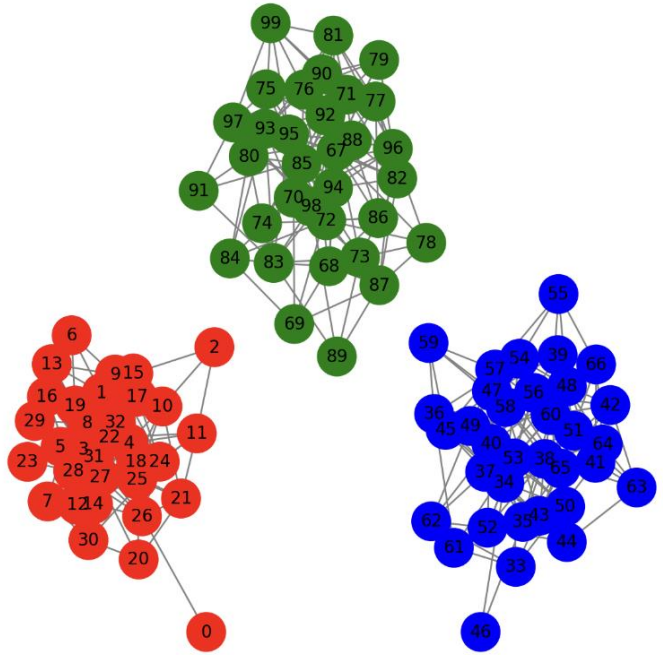
Finding user clusters for targeted advertising and engagement

Detecting fraudulent activity

...

How to detect communities

- Disconnected communities
- Find each components

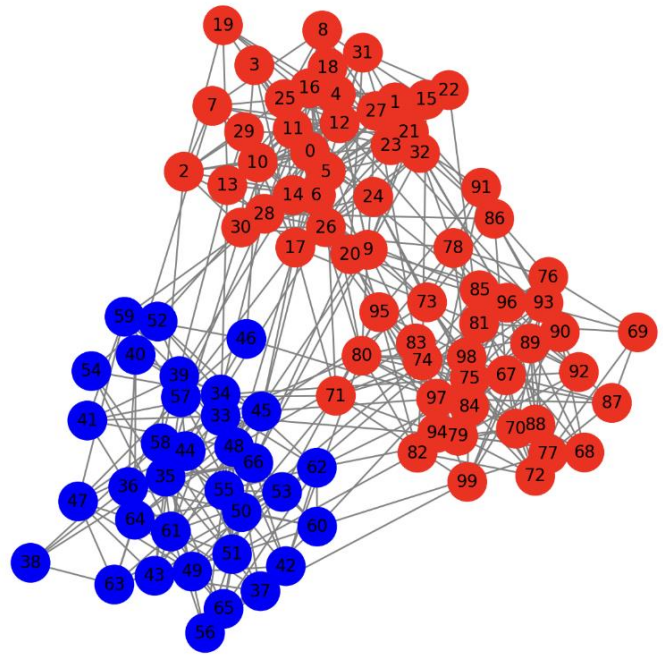


Python

```
import networkx as nx
import pandas as pd

# Load the edge list from CSV
file_path = "disconnected_graph_100_nodes.csv"
df = pd.read_csv(file_path)

# Create a graph from the edge list
G = nx.Graph()
G.add_edges_from(df.values)

# Detect connected components in the previously created unconnected graph (100 nodes)
components_detected = list(nx.connected_components(G))

# Display the detected components
for i, component in enumerate(components_detected):
    print(f"Component {i+1}: {sorted(component)}")
```

Connected communities

- More densely connected internally than with the rest of the network
- Intuitive idea: Prune / wash off sparsely connected parts of the networks

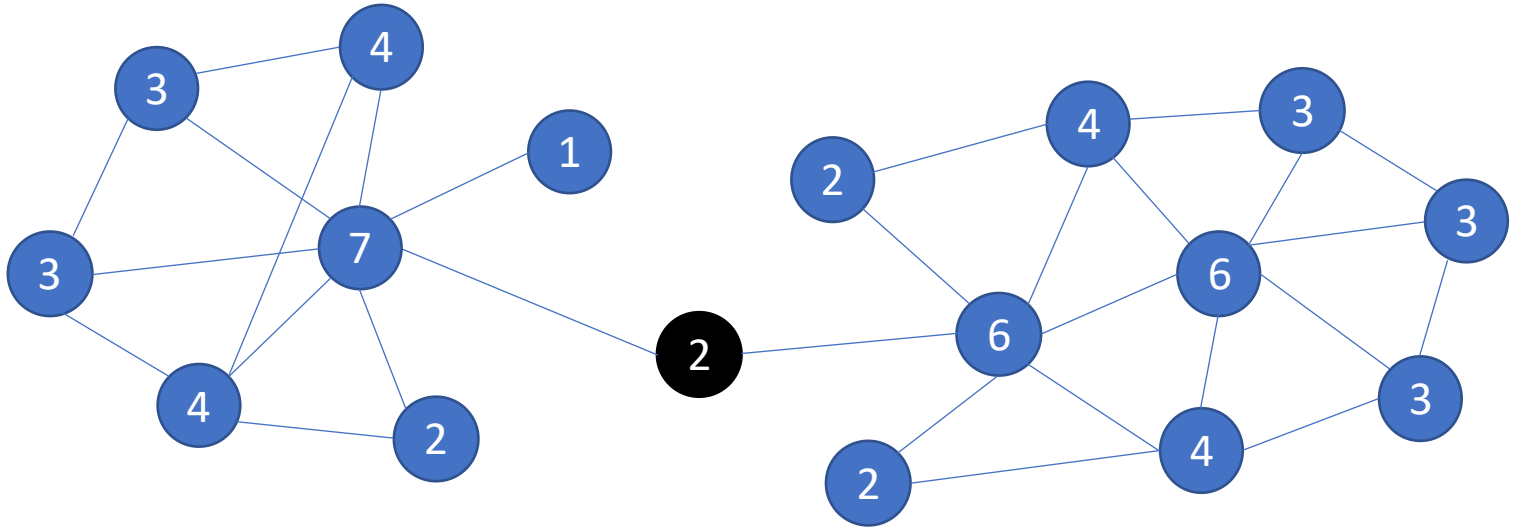


k-Core decomposition

- Goal: identify densely connected communities
- Approach: wash off sparsely connected nodes, i.e., low-degree nodes
- The broker tends to be node with low degree
- Result: A **k-core** is a maximal subgraph where each node has at least k connections

k-Core decomposition

- Choose a k value (starting from k=1).
- Remove all nodes with degree < k.
- Repeat until the remaining nodes all have degree ≥ k.
- Increase k and repeat until the network is fully dissected.

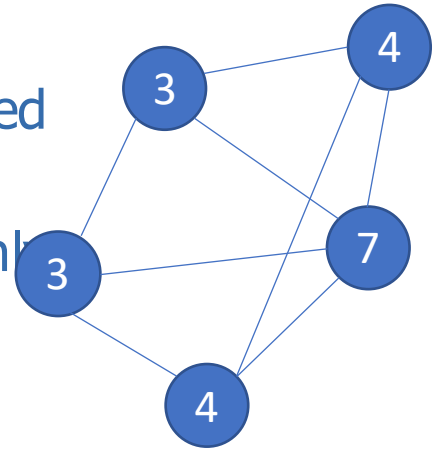


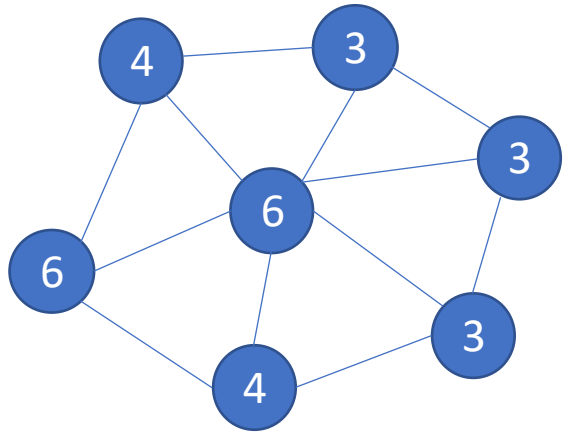
Python

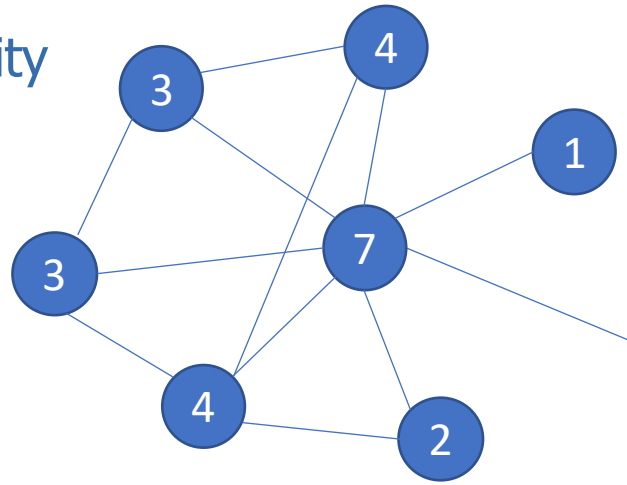
```
k_core = nx.k_core(G, k=2)  # k=2 means every node must have at least 2 connections
components_in_k_core = list(nx.connected_components(k_core))

# Print k-core nodes
print("Nodes in the 2-core:", list(k_core.nodes()))

# Display the detected components
for i, component in enumerate(components_in_k_core):
    print(f"Component {i+1}: {sorted(component)}")
```

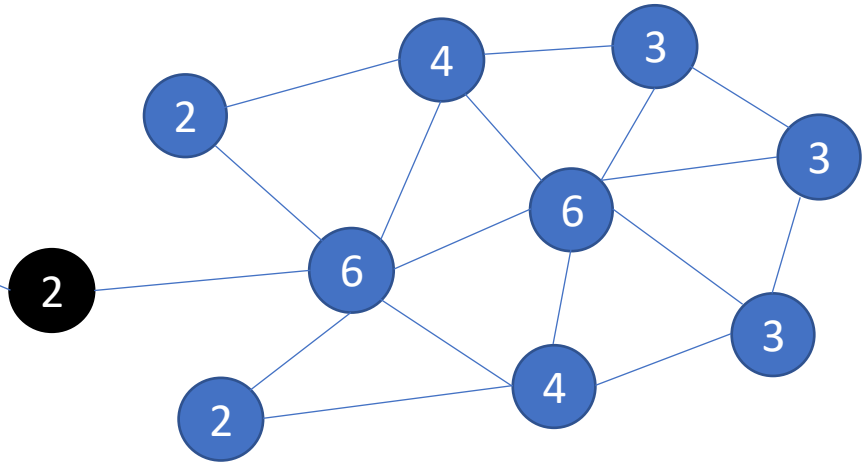
Functionality

- Identifies the most densely connected and robust parts of the network
- The "backbone" of the network – only the strongest connections remain
- You can choose k to your discretion
- Not for dissecting a network
- Limitation: may wash off the majority of the networks and identify 1 or even 0 community



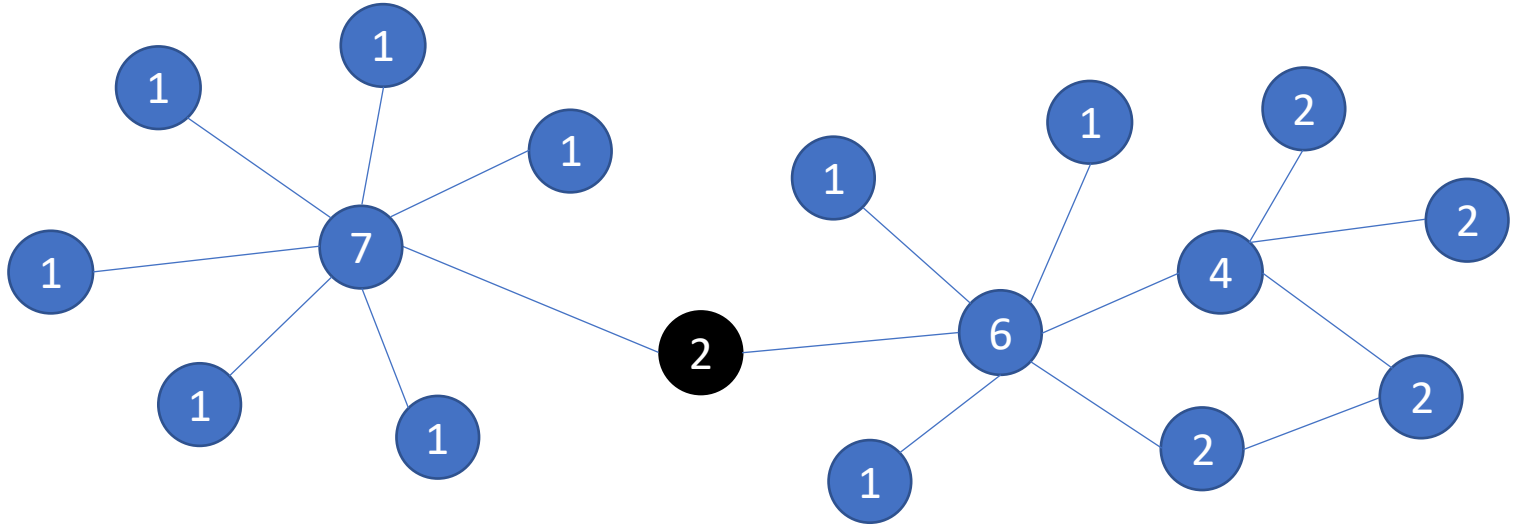






Remove edges, not nodes

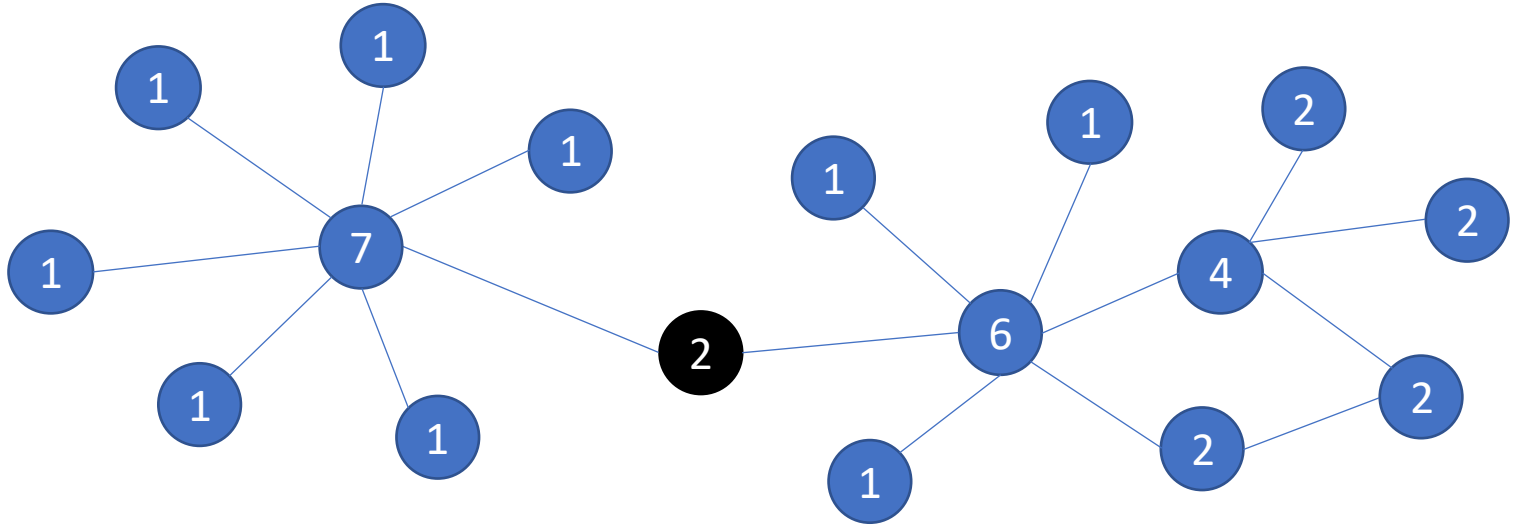
- Intuitive idea: Remove the “broker” type of edges
- What kind of edges to remove?
- Highest betweenness centrality
- These edges often serve as "bridges" between different communities.



Girvan-Newman algorithm

- Progressively removing edges with the highest betweenness centrality
- Process:
  - Compute the betweenness centrality of all edges.
  - Remove the edge with the highest centrality.
  - Recalculate edge betweenness and repeat until the graph breaks into communities.
- Limitation: heavy computation, not scalable for large networks

Python

- ```
  ##connected
  import networkx as nx
  import pandas as pd
  from networkx.algorithms.community import girvan_newman

  # Load the edge list from CSV
  file_path = "connected_graph_100_nodes.csv"
  df = pd.read_csv(file_path)

  # Create a graph from the edge list
  G = nx.Graph()
  G.add_edges_from(df.values)

  # Apply Girvan-Newman Algorithm
  comp = girvan_newman(G)
  top_level_communities = next(comp)  # First split into communities

  # Print detected communities
  for i, community in enumerate(top_level_communities):
      print(f"Community {i+1}: {sorted(community)}")
  ```
  ```

Stoer-Wagner algorithm

- Connectivity: the minimal number of edges/nodes to dissect a network
- Identify and remove such edges
- Stoer-Wagner algorithm finds the global minimum cut, meaning the smallest set of edges that disconnects any part of the network.
- Limitation: most likely to be only 2 communities that are not necessarily well connected within each



Python

- ```
  ##Stoer-Wagner algorithm
  cut_value, partition = nx.stoer_wagner(G)

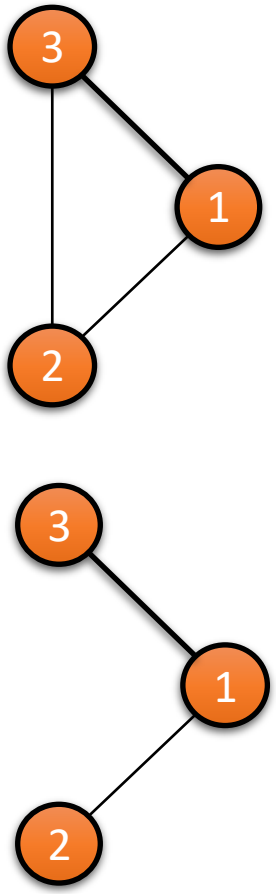
  # Print results
  print("Minimum Cut Size:", cut_value)
  for i, component in enumerate(partition):
      print(f"Component {i + 1}: {sorted(component)}")
  ```
  ```

Louvain algorithm

- A hierarchical method for community detection in networks.
- Based on modularity optimization
- Modularity: measures the density of connections inside communities compared to connections between them.

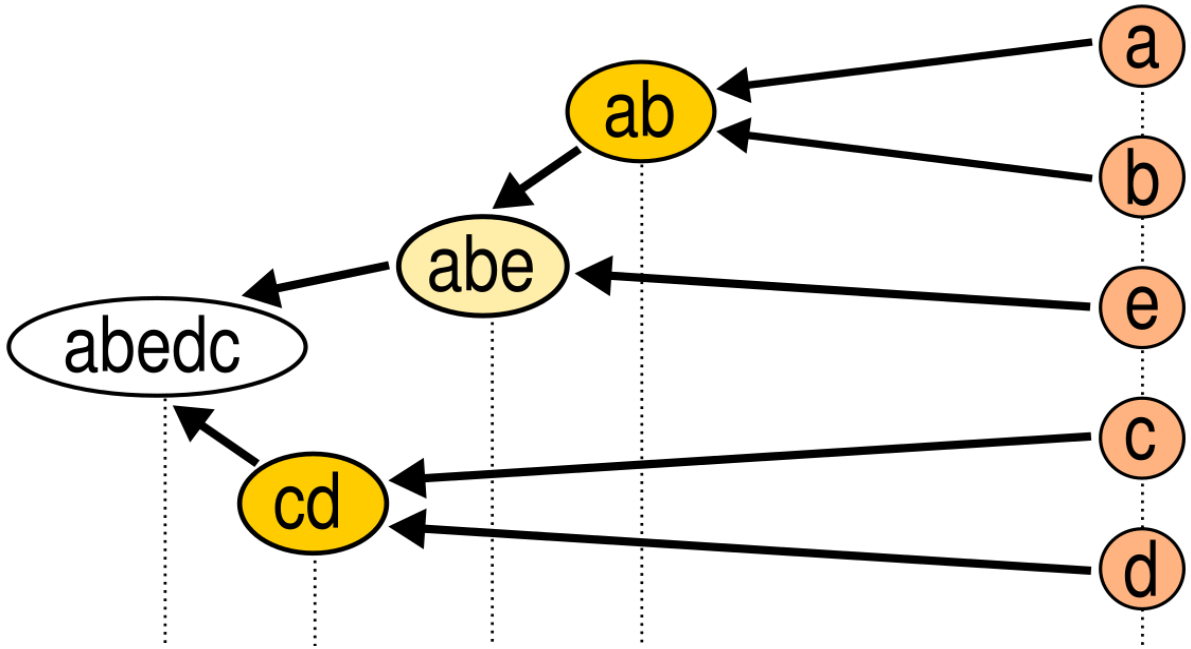
$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

- **A_ij**: Adjacency matrix (1 if edge exists, 0 otherwise).
- **k_i, k_j**: Degrees of nodes **i** and **j**.
- **m**: Total number of edges in the graph.
- **δ(c_i, c_j)**: 1 if nodes **i** and **j** belong to the same community, otherwise 0.



Process

- **Initialization**: Assign each node to its own community.
- **Phase 1 - Modularity Optimization**:
  - Move a node to a neighboring community to maximize modularity.
- **Phase 2 - Community Aggregation**:
  - Collapse detected communities into super-nodes.
- **Repeat** phase 1 and 2 iteratively until no improvement is possible.



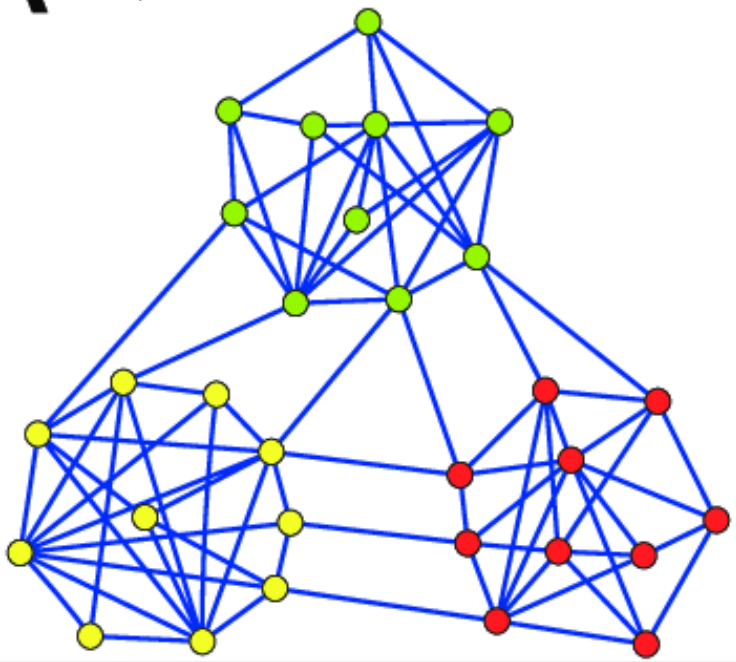
Python

```
# Install using 'pip install python-louvain
import community.community_louvain as community_louvain

# Apply Louvain algorithm for community detection
partition = community_louvain.best_partition(G)

# Print detected communities
communities = {}
for node, comm in partition.items():
    if comm not in communities:
        communities[comm] = []
    communities[comm].append(node)

for comm, nodes in communities.items():
    print(f"Community {comm+1}: {sorted(nodes)}")
```

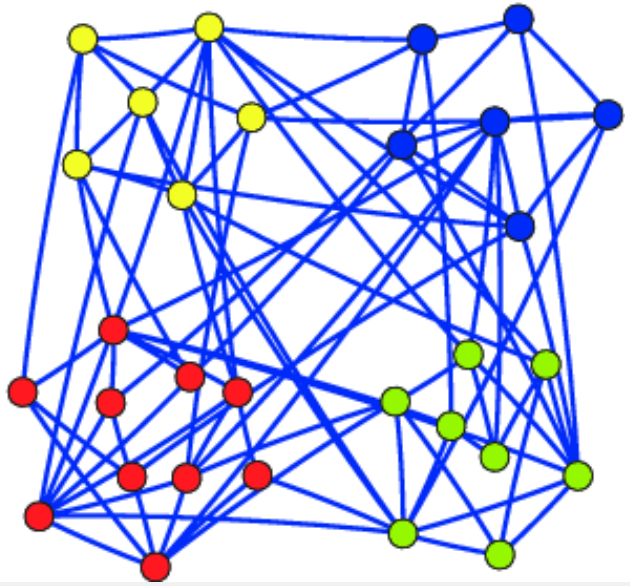
Indicator of network communities

- While you can always apply network partitioning
- It doesn't mean the partitioning results make good sense
- This can be reflected by the final modularity score

A Q=0.7



B Q=0.2



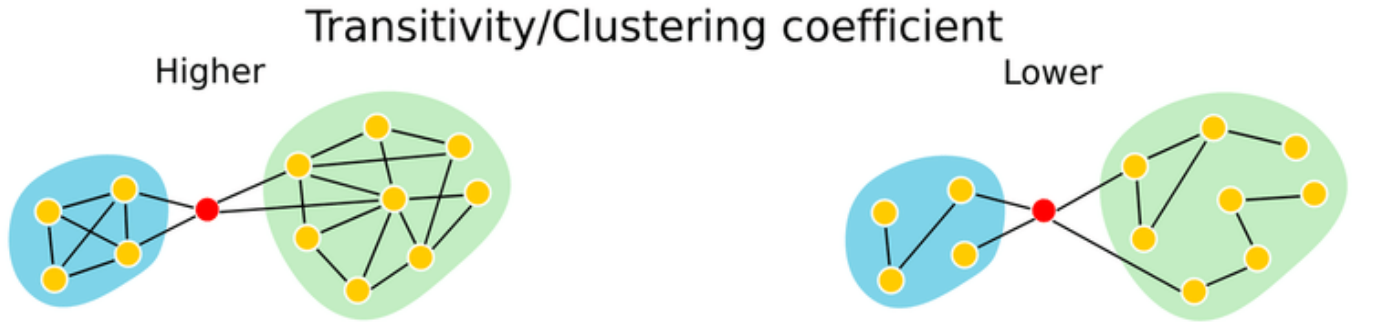
Modularity score

- Post-partitioning modularity scores

Modularity (Q)	Community Structure Strength
$Q < 0.1$	No significant communities (almost random graph).
$0.1 \leq Q < 0.3$	Weak community structure.
$0.3 \leq Q < 0.5$	Moderate to strong community structure.
$Q \geq 0.5$	Well-defined, strong communities.
$Q \approx 1$	Highly modular, almost disconnected groups.

Pre-partitioning indicators

- Cohesion



- Centralization

Centralization Type	Measures	Key Question
Degree Centralization	Node Degree	Is the network controlled by a few well-connected nodes?
Betweenness Centralization	Shortest Paths	Do a few nodes act as key brokers?
Closeness Centralization	Distance to Others	Are some nodes much closer to all others?
Eigenvector Centralization	Influence	Do a few nodes dominate in influence?

MGP 6a

- For your good performance, your client want to collaborate with you on a new campaign. This campaign launches some community-level promotion. In specific, they want to find some communities and then design promotional strategies that are tailored to them. (e.g., differentiated strategies for Chinese, Turkish, Italian, or Vietnamese communities, knowing that they all have fair amount of presence in the scope of networks)
- Among the two networks, which one may better fit this strategy?
- Please help the campaign manager to find such communities using Girvan-Newman, Stoer-Wagner, and Louvain methods

Roadshow day

- Order: see the `Vote' section
- 5-min lightening talk, be brief, highlight the merits, not necessarily everyone
- Vote 2 fav teams by the end of the day
- Peer evaluation launched

Final quiz

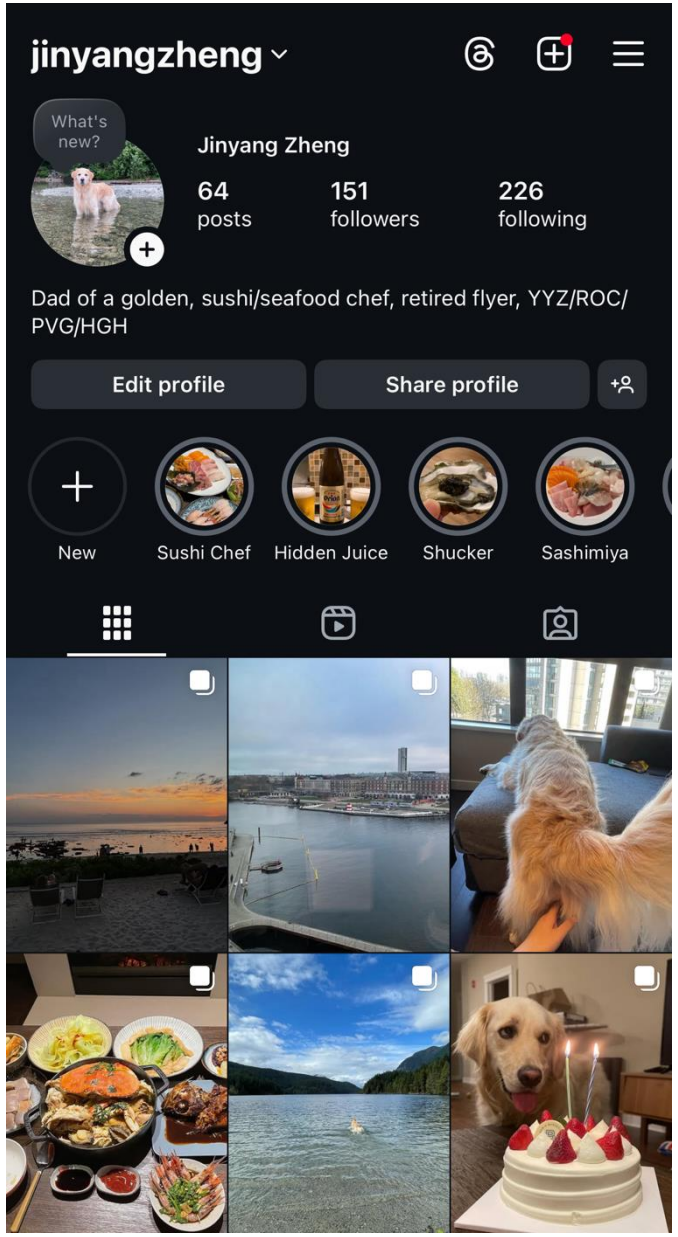
- 70 multiple-choice questions, 90 mins
- Pure concepts, simple computations, completing simple coding (no syntax)
- See the scope and exemplar questions in the review materials, as well as previous warm-up quizzes

Reminders

- Please complete peer evaluation
- Please complete course evaluation

Thank you for joining us!

- This is the trend and frontiers of analytics and AI!
- Thank you for challenging yourself.
- Lifetime warranted
- Good lucks!!
- Please give us a review and rating!!!




MELIORA